

Research Note: Low-Level Design (LLD) Practice & Evaluation Platform

Author: Mainak Pal

Project: CipherSchools Engineering Assignment

Timeline: 2-Day Engineering MVP

Topic: Designing a Meaningful Practice Loop for Low-Level System Design

1. Executive Summary & The Learner Problem

Practicing Low-Level Design (LLD / Object-Oriented Design) is a notorious bottleneck for software engineers transitioning from intermediate coding to senior design maturity. While Data Structures and Algorithms (DSA) benefit from standardized platforms with instant, deterministic unit testing (e.g., LeetCode, Codeforces), LLD practice remains fragmented, subjective, and frustrating.

When a learner prepares for an LLD problem—such as designing a **Parking Lot**, **Elevator Control System**, **Vending Machine**, or **Rate Limiter**—they face five acute friction points:

1. How do learners currently practice?

Learners typically grab a notebook or Google Doc, sketch a few class names, peek at a tutorial or YouTube video after 15 minutes, and copy the author's structure. There is virtually no active retention, failure recovery, or rigorous trial-and-error.

2. How does the learner decide whether their solution is good?

Without an experienced staff engineer reviewing their code, learners have no reliable feedback mechanism. Unlike algorithms where output is either correct or incorrect, low-level design is an exercise in trade-offs, separation of concerns, and clean abstraction. Learners cannot tell whether their 8 classes are over-engineered or missing key polymorphic extension points.

3. What happens when two valid designs look very different?

In LLD, there is rarely a single "canonical" solution. One engineer may model a Parking Lot with a strategy-driven spot allocator (`IParkingStrategy`) and separate hourly billing policies (`IBillingPolicy`), while another utilizes an event-driven observer pattern. Current resources often treat a single author's diagram as the "correct" answer, falsely penalizing legitimate alternative designs.

4. What feedback actually helps a learner improve on the next attempt?

Generic praise ("Looks good!") or unanchored scores ("7/10") do not teach. Useful feedback must be **anchored to concrete evidence** in the submission: identifying where the Single Responsibility Principle was violated, pointing out high coupling between entities, warning about missing concurrency guards, and proposing a targeted next step for the next iteration.

5. What evidence should the platform retain from an attempt?

To foster genuine mastery, the platform must preserve the learner's assumptions, class

definitions, relationships, and design trade-offs across attempts. It must compute the **progression delta** between Attempt \$N\$ and Attempt \$N+1\$, demonstrating whether the candidate successfully resolved earlier critique.

2. Research of Existing Approaches & Market Landscape

To inform our MVP, we researched four prominent existing paradigms for software design practice:

Approach / Platform	Typical Workflow	Strengths	Critical Gaps for LLD Learners
DSA Platforms (LeetCode, HackerRank, NeetCode)	Code editor → Run unit tests → Instant Pass/Fail.	Deterministic, high trust, fast feedback loop, clear objective milestones.	Zero applicability to design thinking. DSA tests inputs vs. outputs; it cannot evaluate encapsulation, cohesion, pattern appropriateness, or extensibility.
Interactive Textbooks (Educative Grokking LLD, Arpit Bhayani, Design Gurus)	Read chapter → Review author's UML diagram → Inspect reference Java/C++ code.	High educational quality, explains common patterns, provides curated interview questions.	Passive consumption. The learner does not construct designs from scratch, does not encounter design edge cases, and receives no feedback on their own variations.
Generic LLM Chatbots (ChatGPT, Claude, Gemini Web UI)	Paste prompt: "Review my parking lot design" → Receive conversational response.	Accessible, flexible, capable of reasoning about object relationships and edge cases.	Unconstrained and inconsistent. LLMs tend to offer flattering, vague reviews without a rubric; they hallucinate scores, drift into High-Level Design (e.g. suggesting Kafka/Kubernetes for a parking lot), and fail to track attempt history systematically.
Open Source Repositories (GitHub awesome-low-level-design, Gaurav Sen, etc.)	Clone repo → Browse class implementations.	Real code examples in Java/Python/C++.	Static code dumps. No guidance, no verification, no deliberate practice loop.

3. Key Gaps Identified

From this landscape research, three foundational gaps emerge:

Gap A: The "Binary vs. Unconstrained" Evaluation Dilemma

Platforms either attempt to force LLD into brittle unit test harnesses (which forces learners to match exact class names and method signatures) or rely on unconstrained AI prompts that produce inconsistent, hand-waving critiques.

Insight: Evaluation must be **rubric-driven and hybrid**—combining deterministic structural checks with structured, evidence-anchored AI reasoning.

Gap B: Conflation of LLD with High-Level Distributed Systems

Novice learners and poorly-prompted LLMs constantly confuse LLD with HLD. An LLD attempt for an Elevator often devolves into discussions of WebSocket gateways, Redis clusters, and load balancers, rather than core object modeling: dispatching strategies, elevator state patterns, internal vs. external button models, and thread safety.

Insight: The platform must firmly enforce **LLD scope boundaries** through structured submission forms (Requirements & Assumptions, Entity Hierarchy, Design Patterns, Concurrency/Edge Cases).

Gap C: Absence of an Iterative "Attempt Progression" Loop

Existing tools treat design as a one-shot exam. In reality, software engineering design reviews are inherently iterative: a design is presented, reviewed, revised, and re-submitted.

Insight: An LLD platform's killer feature is **Attempt Progression Tracking**—allowing the learner to re-attempt a problem and see an explicit diff: *"In Attempt 1, you coupled Spot Allocation with Payment. In Attempt 2, you extracted `IParkingStrategy`, raising your Abstraction score from 2/5 to 5/5."*

4. How Industry Evaluators Actually Evaluate LLD

In senior engineering interviews (Google, Uber, Amazon, Atlassian), interviewers evaluate low-level design along four specific pillars:

1. **Change Tolerance (Open/Closed Principle):** If the interviewer asks, *"Now support electric vehicle charging spots,"* does the candidate need to rewrite their core `ParkingLot` class, or does their polymorphic abstraction absorb the change seamlessly?
2. **State vs. Behavior Decoupling:** Are entities modeled with single responsibilities, or is there an all-knowing "God Object"?
3. **Concurrency & Race Conditions:** When two cars enter at the same second, does the code address synchronization (locks, thread-safe collections)?
4. **Clarity of Assumptions:** Did the candidate bound the problem scope, or did they assume infinite capacity?

Our platform's evaluation rubric and structured starter templates are tailored specifically to train learners on these exact industry competencies.

5. Product Direction & MVP Scope

Our MVP focuses strictly on the **Learner Practice Loop**:

Choose Problem → Think & Design → Submit → Rubric Feedback → Review → Iterate (Try Again)

Core Decisions for the MVP:

1. Curated Problem Library (4 Archetypal Problems):

- **Parking Lot System** (Inheritance, Strategy Pattern for allocation & pricing, concurrency).
- **Elevator Control System** (State Pattern, SCAN/LOOK Scheduling Algorithm, Dispatcher).
- **Vending Machine** (Classic State Pattern: Idle, HasMoney, Dispensing, SoldOut, transaction rollback).
- **Rate Limiter Library** (Token Bucket, Leaky Bucket, Sliding Window, Thread-safe interface design).

2. Structured Submission Format: A hybrid model capturing **Assumptions, Entity & Interface Skeleton, Mermaid Class Diagram, and Trade-offs / Edge Cases**. This yields high design evidence without requiring heavy language compilers.

3. Pluggable Evaluation Pipeline:

- **Deterministic Layer**: Validates submission completeness, presence of core entities, interface definitions, and edge case coverage.
- **Cognitive / Rubric Layer**: Evaluates single responsibility, coupling/cohesion, appropriate design patterns, and extensibility. Every score must cite specific **Evidence, Concerns**, and **Actionable Suggestions**.

4. Resilient Asynchronous State Engine: Submissions transition through **SUBMITTED** → **EVALUATING** → **COMPLETED** / **FAILED**. Even if external AI is delayed or errors out, learner work is safely persisted and can be re-evaluated idempotently.

5. Attempt History & Visual Delta: Learners can view past attempts side-by-side, inspect their score progression, and verify how their architectural decisions matured over time.